



ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

**Σχεδιασμός και Υλοποίηση Web Συστήματος Παραγγελιοληψίας και Διαχείρισης
Λογαριασμών για Χώρους Εστίασης με ASP.NET Core και SQLite**

Ματθαίος Δρετάκης

Εισηγητής: Δημήτρης Πλεξουσάκης, Καθηγητής

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

**Σχεδιασμός και Υλοποίηση Web Συστήματος Παραγγελιοληψίας και Διαχείρισης
Λογαριασμών για Χώρους Εστίασης με ASP.NET Core και SQLite**

Ματθαίος Δρετάκης

A.M: 3357

Εισηγητής:

Δημήτρης Πλεξουσάκης, Καθηγητής

ΠΕΡΙΛΗΨΗ

Η παρούσα εργασία παρουσιάζει τον σχεδιασμό και την υλοποίηση του **BarakiBar Web PDA**, ενός ελαφρού web συστήματος παραγγελιοληψίας και διαχείρισης λογαριασμών για μικρού μεγέθους χώρους εστίασης. Η λύση βασίζεται σε αρχιτεκτονική client-server, όπου ένα frontend που εκτελείται στον browser επικοινωνεί με backend ASP.NET Core Web API μέσω HTTP endpoints. Τα δεδομένα του μενού και οι ποσότητες αποθέματος αποθηκεύονται μόνιμα σε τοπική βάση **SQLite** με χρήση του **Entity Framework Core**, προσφέροντας εύκολη εγκατάσταση χωρίς ανάγκη ξεχωριστού database server. Το σύστημα υποστηρίζει διαχείριση τραπεζιών και “bar tabs”, αναλυτικούς λογαριασμούς, μερικές και ολικές πληρωμές, φιλοδωρήματα και εκπτώσεις, καθώς και βασική συγκεντρωτική εικόνα “ταμείου”. Η εργασία αναδεικνύει ότι ένα μικρό και απλό τεχνολογικό στοίβασμα (HTML/CSS/JavaScript + ASP.NET Core + SQLite/EF Core) μπορεί να καλύψει ρεαλιστικές λειτουργικές ανάγκες με χαμηλή πολυπλοκότητα και να λειτουργεί αξιόπιστα σε τοπικό δίκτυο ή offline περιβάλλον. Στην τελική έκδοση προστέθηκαν επίσης μηχανισμός συγχρονισμού πολλαπλών clients σε πραγματικό χρόνο μέσω SignalR, διαχειριστικό περιβάλλον μενού με προστασία κωδικού πρόσβασης, καθώς και προσαρμοσμένη mobile-friendly διεπαφή για χρήση από κινητά και tablets.

ABSTRACT

This project implements a lightweight Point-of-Sale (POS) web application, designed for a small bar environment, called **BarakiBar Web PDA**. The system enables staff to manage tables, add menu items to orders, track pending balances, and close bills using cash or card payments while supporting tips and discounts. A key requirement was to keep the solution simple, locally deployable, and maintainable with minimal infrastructure. The implementation uses an **ASP.NET Core Web API** backend, a **vanilla HTML/CSS/JavaScript** frontend, and **SQLite + Entity Framework Core** for persistent storage of menu items and stock availability. The solution follows a service-oriented design where the frontend remains thin and interacts with the backend exclusively through HTTP endpoints. The final version also includes real-time synchronization between multiple clients through SignalR, a password-protected admin interface for menu management, and a mobile-friendly responsive user interface for smartphone and tablet use.

ΕΠΙΣΤΗΜΟΝΙΚΗ ΠΕΡΙΟΧΗ: Ανάπτυξη Λογισμικού

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: ASP.NET Core, SQLite, Entity Framework Core, SignalR, Web Application, Responsive UI

ΠΕΡΙΕΧΟΜΕΝΑ

Table of Contents

Κεφάλαιο 1 — Εισαγωγή	7
Κεφάλαιο 2 — Στόχοι, Απαιτήσεις και Περιορισμοί.....	9
Κεφάλαιο 3 — Τεχνολογίες και Εργαλεία.....	11
Κεφάλαιο 4 — Συνολική Αρχιτεκτονική και Σχεδιασμός	13
Κεφάλαιο 5 — Μοντελοποίηση Δεδομένων και Σχήμα Βάσης.....	15
Κεφάλαιο 6 — Υλοποίηση Backend: ASP.NET Core Web API και Επιχειρησιακή Λογική	17
Κεφάλαιο 7 — Υλοποίηση Frontend: UI Λογική, Απόδοση και API Κλήσεις	20
Κεφάλαιο 8 — Δοκιμές, Σενάρια Χρήσης και Επαλήθευση Λειτουργικότητας.....	26
Κεφάλαιο 9 — Συμπεράσματα, Περιορισμοί και Μελλοντικές Επεκτάσεις	28
ΠΑΡΑΡΤΗΜΑ Α — Ενδεικτικά API Endpoints και Παραδείγματα Κλήσεων (REST).....	29
A.1 Γενικές αρχές επικοινωνίας	29
A.2 Tables API.....	29
A.2.1 Λίστα τραπεζιών	29
A.2.2 Λεπτομέρειες συγκεκριμένου τραπεζιού	30
A.2.3 Δημιουργία νέου bar table	31
A.2.4 Ονομασία τραπεζιού (όνομα πελάτη).....	31
A.2.5 Προσθήκη είδους στο τραπέζι (με έλεγχο αποθέματος)	31
A.2.6 Κλείσιμο τραπεζιού (ολική πληρωμή)	32
A.2.7 Μερική πληρωμή επιλεγμένων ειδών	33
A.2.8 Αφαίρεση επιλεγμένων ειδών (χωρίς πληρωμή / ακύρωση)	34
A.2.9 Μεταφορά λογαριασμού σε άλλο τραπέζι	35
A.3 Menu API (SQLite μέσω EF Core).....	35
A.3.1 Λίστα ενεργών ειδών (PDA Menu)	35
A.3.2 Λίστα όλων των ειδών (Admin Menu).....	36
A.3.3 Λίστα κατηγοριών.....	36
A.3.4 Προσθήκη νέου item	36
A.3.5 Ενημέρωση item (τιμή/stock/active).....	37

A.3.6 Hard delete item (οριστική διαγραφή)	37
A.4 Register API	37
A.4.1 Κατάσταση ταμείου και εκκρεμοτήτων.....	37
A.4.2 Close register (reset με προϋπόθεση pending=0).....	38
A.4.3 Επαλήθευση πρόσβασης στο Admin UI	38
ΠΑΡΑΡΤΗΜΑ Β — SQLite / EF Core: Σχήμα, Παραδείγματα SQL και Ερμηνεία Αποτελεσμάτων	39
B.1 Τοπική αποθήκευση με SQLite	39
B.2 Entity MenuItem και αντιστοίχιση στον πίνακα MenuItems	39
B.3 Ενδεικτικό DDL δημιουργίας πίνακα	39
B.4 Πρακτικά SQL queries για έλεγχο και αναφορές.....	40
B.5 Ερμηνεία EF Core queries και ροή ενημέρωσης stock	41
B.6 Διαγνωστικός έλεγχος σωστής λειτουργίας της βάσης.....	41
ΠΑΡΑΡΤΗΜΑ Γ — Τελική Ροή Χρήσης και Διαχείριση Μενού/Αποθέματος στην Τελική Έκδοση	43
Γ.1 Περιγραφή τελικής εμπειρίας χρήσης (end-to-end).....	43
Γ.2 Τελική αρχιτεκτονική ροή δεδομένων (UI → API → SQLite/State → UI).....	44
Γ.3 Τελική διαχείριση μενού και αποθέματος μέσω Admin UI (Save All, αναζήτηση, hard delete)	45
ΠΑΡΑΡΤΗΜΑ Δ — Τεχνική Επαλήθευση, Troubleshooting και Έλεγχοι Ορθής Λειτουργίας	46
Δ.1 Έλεγχος συγχρονισμού UI και API (consistency checks).....	46
Δ.2 Έλεγχος λειτουργίας SQLite (schema, seeding, CRUD)	46
Δ.3 Caching σε browser και αποφυγή “φανταστικών” προβλημάτων UI	46
Δ.4 Έλεγχος σωστής διάκρισης /api/menu vs /api/menu/all	47
Δ.5 Έλεγχος κανόνα “Close register μόνο όταν pending=0”	47
Δ.6 Έλεγχος ακύρωσης/διόρθωσης παραγγελίας (Remove selected)	47
Δ.7 Έλεγχος real-time συγχρονισμού πολλών clients.....	47
Δ.8 Έλεγχος responsive/mobile λειτουργίας και mobile navigation bar	48

Κεφάλαιο 1 — Εισαγωγή

Η ψηφιοποίηση βασικών διαδικασιών στον χώρο της εστίασης αποτελεί διαρκή ανάγκη, ειδικά σε μικρές επιχειρήσεις όπου ο χρόνος, η ακρίβεια και η ευκολία χειρισμού επηρεάζουν άμεσα την ποιότητα εξυπηρέτησης και τη συνολική λειτουργία. Σε περιβάλλοντα όπως bar και καφέ, η διαχείριση πολλών τραπεζιών, η αποτύπωση των παραγγελιών σε πραγματικό χρόνο και το ορθό κλείσιμο λογαριασμών δημιουργούν ένα σύνολο απαιτήσεων που, αν δεν καλυφθούν με πρακτικό τρόπο, οδηγούν σε λάθη ή καθυστερήσεις. Συχνά, οι μικρές επιχειρήσεις καταλήγουν να χρησιμοποιούν πρόχειρες λύσεις (χειρόγραφες σημειώσεις ή απλές λίστες), οι οποίες δεν παρέχουν συνέπεια, ιστορικό, ασφαλή υπολογισμό ποσών, ούτε έλεγχο αποθέματος.

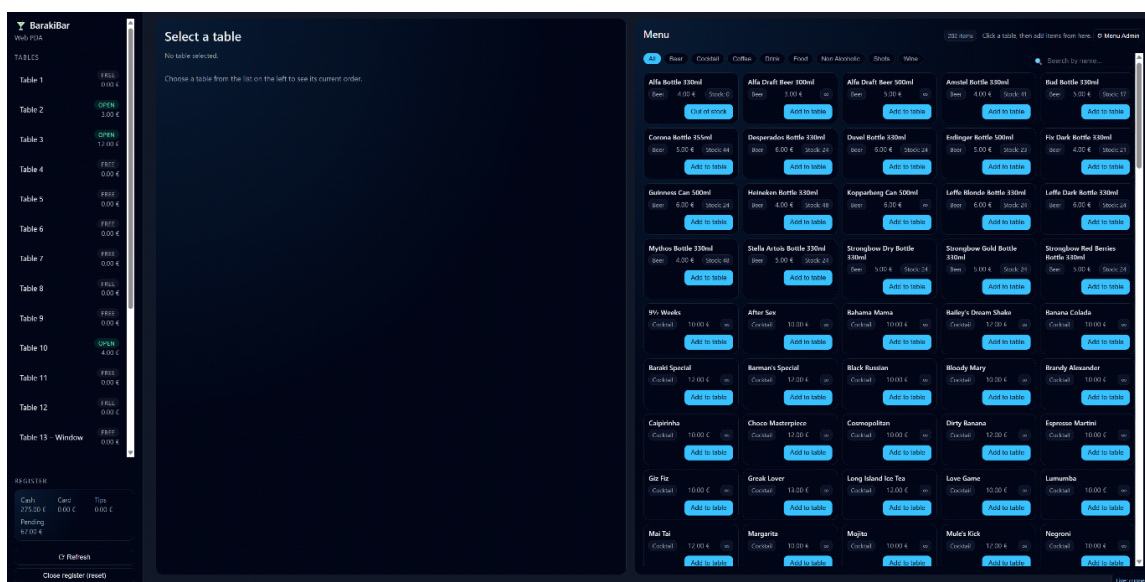
Στο πλαίσιο αυτό, η παρούσα εργασία στοχεύει στη δημιουργία ενός web συστήματος τύπου “PDA”, το οποίο μπορεί να εκτελείται σε οποιοδήποτε σύγχρονο browser. Η επιλογή web προσέγγισης έχει πρακτικά πλεονεκτήματα: δεν απαιτεί εγκατάσταση εφαρμογής σε κάθε συσκευή, υποστηρίζεται εύκολα από κινητό/τάμπλετ/PC και μπορεί να λειτουργεί μέσα σε τοπικό δίκτυο καταστήματος. Η εφαρμογή σχεδιάζεται με γνώμονα τη χαμηλή πολυπλοκότητα, ώστε να μπορεί να αναπτυχθεί, να λειτουργήσει και να συντηρηθεί χωρίς περίπλοκες υποδομές.

Κεντρικό σημείο της εργασίας αποτελεί ο τρόπος αποθήκευσης και διαχείρισης των δεδομένων, ειδικά για το μενού και τα αποθέματα. Η εφαρμογή πρέπει να θυμάται κατηγορίες, τιμές, ενεργά/ανενεργά προϊόντα και ποσότητα αποθέματος για είδη που είναι περιορισμένα. Η SQLite επιλέγεται ως ιδιαίτερα ταιριαστή λύση για μικρά συστήματα με τοπική λειτουργία, διότι προσφέρει επίμονη αποθήκευση σε αρχείο, αποφεύγει την ανάγκη ξεχωριστού database server και διευκολύνει τη μεταφορά/backup της βάσης.

Κατά την εξέλιξη του έργου προστέθηκε και μια κρίσιμη λειτουργία “Menu Admin” μέσα από το ίδιο UI (σε μορφή modal), η οποία επιτρέπει πλήρη διαχείριση του μενού με μαζική αποθήκευση (“Save all”), αναζήτηση και άμεση εγγραφή στη βάση bar.db. Η προσθήκη αυτού του υποσυστήματος κάνει το σύστημα περισσότερο ολοκληρωμένο, καθώς πλέον ο χρήστης δεν χρειάζεται εξωτερικά εργαλεία ή χειροκίνητη SQL για να ενημερώσει προϊόντα, τιμές ή stock.

Κατά την τελική φάση ανάπτυξης, η εφαρμογή εξελίχθηκε πέρα από ένα απλό web interface παραγγελιοληψίας και απέκτησε δυνατότητες ταυτόχρονης χρήσης από πολλαπλές συσκευές, συγχρονισμού αλλαγών σε πραγματικό χρόνο, καθώς και ειδικής προσαρμογής για κινητές συσκευές. Έτσι, το έργο προσεγγίζει περισσότερο μια ολοκληρωμένη εφαρμογή μικρής κλίμακας παρά ένα απλό ακαδημαϊκό πρωτότυπο.

Η εργασία οργανώνεται σε κεφάλαια που καλύπτουν στόχους και απαιτήσεις, αρχιτεκτονική, σχεδιασμό δεδομένων, υλοποίηση API, υλοποίηση διεπαφής χρήστη, δοκιμές και αξιολόγηση, καθώς και περιορισμούς και προτάσεις για μελλοντική επέκταση. Συμπληρωματικά, παρέχονται παραρτήματα με ενδεικτικά API endpoints και αναλυτικότερη επεξήγηση της SQLite/EF Core προσέγγισης.



Εικόνα 1.1: Η βασική διεπαφή του BarakiBar Web PDA σε περιβάλλον desktop.

Κεφάλαιο 2 — Στόχοι, Απαιτήσεις και Περιορισμοί

Ο πρωταρχικός στόχος του έργου είναι να δημιουργηθεί ένα λειτουργικό σύστημα παραγγελιοληψίας και διαχείρισης λογαριασμών, με τρόπο που να προσομοιάζει καθημερινές πρακτικές ενός bar. Η εφαρμογή πρέπει να “στέκεται” επιχειρησιακά: ο χρήστης οφείλει να βλέπει άμεσα σε ποια τραπέζια υπάρχουν ανοιχτοί λογαριασμοί, να επιλέγει τραπέζι και να προσθέτει είδη από ένα οργανωμένο μενού, να κλείνει λογαριασμούς ή να κάνει μερικές πληρωμές με λίγα βήματα, και να έχει ξεκάθαρη εικόνα της ταμειακής κατάστασης μέσω register.

Από λειτουργικής πλευράς, οι απαιτήσεις οργανώνονται σε βασικούς άξονες. Στον άξονα διαχείρισης τραπεζιών, η εφαρμογή υποστηρίζει σταθερά τραπέζια (π.χ. 1–14) και δυναμικά “bar tables” για πελάτες στον πάγκο, με δυνατότητα ονομασίας πελάτη και δυνατότητα μεταφοράς λογαριασμού από τραπέζι σε τραπέζι. Στον άξονα διαχείρισης μενού, το μενού φορτώνεται από επίμονη αποθήκευση (SQLite), εμφανίζεται οργανωμένο ανά κατηγορία, παρουσιάζει τιμή και κατάσταση αποθέματος, και επιτρέπει στο σύστημα να αποτρέπει προσθήκες όταν το προϊόν είναι out-of-stock.

Στον άξονα διαχείρισης λογαριασμών και πληρωμών, κάθε τραπέζι διαθέτει bill με λίστα items και τρέχον σύνολο, δυνατότητα έκπτωσης ποσοστού και καταγραφής φιλοδωρήματος, καθώς και πληρωμή είτε συνολική είτε μερική επιλεγμένων ειδών. Παράλληλα, επιλύθηκε ένα πρακτικό επιχειρησιακό πρόβλημα: όταν ο χειριστής προσθέσει κατά λάθος items σε τραπέζι, πρέπει να υπάρχει τρόπος ακύρωσης. Αυτό υλοποιείται με λειτουργία “Remove selected” (χωρίς πληρωμή), η οποία αφαιρεί items από το bill. Σε συνδυασμό με stock-aware λογική, η ακύρωση μπορεί να συνδέεται με επιστροφή αποθέματος, ώστε ένα λάθος “add” να μην καταναλώνει άδικα stock.

Στον άξονα ταμείου (register), το σύστημα εμφανίζει σύνολα μετρητών, κάρτας, tips και pending (εκκρεμότητες). Επιπλέον, προστέθηκε λειτουργία “Close register”, η οποία επιτρέπει reset των cash/card/tips μόνο όταν το pending είναι μηδενικό. Αυτό αποτελεί σημαντικό κανόνα ασφαλείας: αποφεύγεται να “κλείσει” το ταμείο ενώ υπάρχουν ανοιχτά χρέη.

Στις μη λειτουργικές απαιτήσεις, δίνεται έμφαση στην απλότητα εγκατάστασης, την ταχύτητα απόκρισης και τη μείωση ασυνεπειών μεταξύ UI και πραγματικής κατάστασης. Το σύστημα υιοθετεί μοτίβο “ανάκτησης μετά από ενέργεια” (reload affected views) αντί να διατηρεί πολύπλοκο local state στο frontend. Έτσι μειώνονται οι πιθανότητες σφαλμάτων συγχρονισμού και γίνεται πιο αξιόπιστη η λειτουργία, ειδικά όταν υπάρχουν έλεγχοι stock ή κανόνες πληρωμής.

Ως περιορισμός, στην παρούσα φάση μέρος της κατάστασης (π.χ. tables/bills/register) προκύπτει από υφιστάμενη λογική αρχείων/serialization (FileProcessor), ενώ το μενού και το stock διατηρούνται στη βάση SQLite. Η “διπλή” πηγή δεδομένων είναι αποδεκτή ως ενδιάμεσο στάδιο, αλλά αναγνωρίζεται ως σημείο για μελλοντική ενοποίηση, ώστε το σύστημα να γίνει πλήρως consistent μέσα σε ένα κοινό data layer.

Κεφάλαιο 3 — Τεχνολογίες και Εργαλεία

Η εφαρμογή υλοποιείται με τεχνολογίες κατάλληλες για web συστήματα μικρής έως μεσαίας κλίμακας. Στο backend χρησιμοποιείται ASP.NET Core (Web API), το οποίο εκθέτει endpoints μέσω HTTP και επιστρέφει δεδομένα σε JSON. Η επιλογή αυτή προσφέρει οργανωμένο routing, controllers, middleware pipeline και dependency injection, ενώ υποστηρίζει εύκολα σταδιακή επέκταση του συστήματος.

Για την αποθήκευση του μενού και του stock χρησιμοποιείται SQLite. Η SQLite έχει ιδιαίτερη αξία σε σενάρια τοπικής λειτουργίας, καθώς δεν απαιτεί ξεχωριστό database server. Η βάση είναι ένα αρχείο (bar.db) που μπορεί να δημιουργείται και να ενημερώνεται στο runtime. Επιπλέον, προσφέρει ικανοποιητική αξιοπιστία και είναι επαρκής για το μέγεθος δεδομένων και τον ρυθμό ενημερώσεων ενός μικρού καταστήματος.

Η πρόσβαση στη SQLite γίνεται μέσω Entity Framework Core. Το EF Core λειτουργεί ως ORM, επιτρέποντας queries και updates σε μορφή C# (LINQ) χωρίς χειροκίνητη SQL σε κάθε σημείο. Αυτό βελτιώνει τη συντηρησιμότητα και μειώνει πιθανότητες σφαλμάτων, ενώ η ύπαρξη DbContext και entities βοηθά στη σαφή μοντελοποίηση: ο πίνακας MenuItems αντιστοιχεί σε entity MenuItem με πεδία Name, Category, Price, Active και StockQuantity.

Για την υποστήριξη συγχρονισμού μεταξύ πολλαπλών clients χρησιμοποιήθηκε επίσης το SignalR. Ο μηχανισμός αυτός επιτρέπει στο backend να αποστέλλει ειδοποιήσεις σε όλες τις συνδεδεμένες συσκευές όταν μεταβάλλεται η κατάσταση του συστήματος, όπως κατά την προσθήκη προϊόντος, την αφαίρεση item, την ενημέρωση του μενού ή το κλείσιμο λογαριασμού.

Στο frontend χρησιμοποιούνται HTML, CSS και JavaScript χωρίς framework. Η επιλογή αυτή επιτρέπει απλότητα και πλήρη έλεγχο του DOM, ενώ η επικοινωνία με το backend γίνεται μέσω fetch() και μιας κοινής βοηθητικής συνάρτησης fetchJson(). Το UI οργανώνεται ως single-page εμπειρία, με sidebar για τραπέζια και register, κύρια καρτέλα για table details, και καρτέλα μενού σε grid μορφή. Τέλος, ενσωματώθηκε modal “Menu Admin”, το οποίο αποτελεί μέρος του frontend αλλά λειτουργεί ως διαχειριστικό υποσύστημα με δικό του search bar και μαζική αποθήκευση αλλαγών στη βάση. Η διεπαφή επεκτάθηκε με responsive CSS και ειδικές

ρυθμίσεις πλοήγησης για mobile χρήση, ώστε να λειτουργεί πρακτικά και σε smartphone/tablet, τόσο σε portrait όσο και σε landscape mode.

Συνοπτική αρχιτεκτονική του συστήματος



Εικόνα 3.1: Συνοπτική αρχιτεκτονική του συστήματος.

Κεφάλαιο 4 — Συνολική Αρχιτεκτονική και Σχεδιασμός

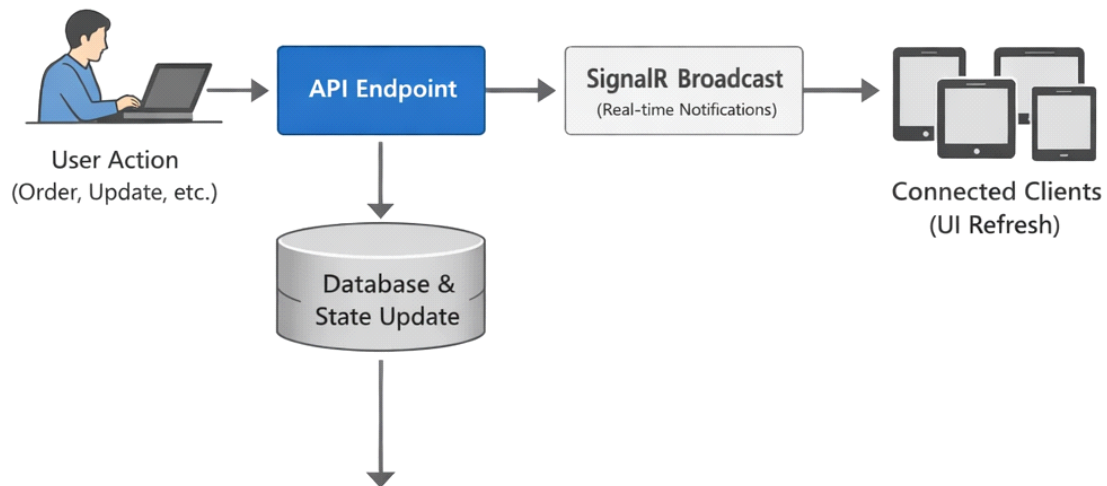
Η εφαρμογή ακολουθεί αρχιτεκτονική client–server. Ο client είναι ο browser, ο οποίος παρουσιάζει τη διεπαφή και εκτελεί ενέργειες μέσω API κλήσεων. Ο server είναι το ASP.NET Core Web API, το οποίο εξυπηρετεί τόσο τα static αρχεία του UI (index.html, styles.css, app.js) όσο και τα endpoints που επιστρέφουν JSON.

Ο διαχωρισμός αυτός είναι κρίσιμος. Η επιχειρησιακή λογική συγκεντρώνεται στον server και ο client δεν θεωρείται πηγή “αλήθειας”. Η επίμονη αποθήκευση γίνεται κεντρικά: το μενού/stock αποθηκεύονται στη SQLite και μόνο το backend πραγματοποιεί τις μεταβολές. Επιπλέον, κανόνες όπως “Out of stock”, “Close register μόνο όταν pending=0” ή “hard delete προϊόντος” υλοποιούνται server-side, ώστε να μην μπορούν να παρακαμφθούν από τον client.

Το UI οργανώνεται σε δύο κύριες περιοχές. Στη sidebar εμφανίζεται η λίστα τραπεζιών με κατάσταση “Open/Free” και τρέχον σύνολο, καθώς και η σύνοψη register με cash/card/tips/pending. Στο κεντρικό μέρος, ο χρήστης βλέπει το επιλεγμένο τραπέζι με αναλυτικό bill, επιλογές πληρωμής (ολική/μερική), επιλογές έκπτωσης και tip, καθώς και δυνατότητα μεταφοράς λογαριασμού. Στη δεξιά περιοχή εμφανίζεται το μενού ως grid, με φίλτρα κατηγορίας και search bar, ώστε η εύρεση προϊόντων να είναι άμεση.

Η ροή δεδομένων υιοθετεί πρακτική λογική “ανάκτησης μετά από ενέργεια”. Όταν ο χρήστης εκτελεί μια ενέργεια που αλλάζει κατάσταση (π.χ. προσθήκη item, πληρωμή, αφαίρεση item), ο client καλεί το backend και στη συνέχεια επαναφορτώνει τα σχετικά τμήματα UI (tables, table details, register, menu). Αυτή η επιλογή αποφεύγει ασυνέπειες, ειδικά όταν υπάρχουν κανόνες stock ή όταν το backend εφαρμόζει πρόσθετους υπολογισμούς (π.χ. multiplier για κάρτα). Στην πράξη, αποδεικνύεται πιο αξιόπιστη από ένα “βαρύ” state management στο frontend.

Στην τελική έκδοση, ο παραπάνω μηχανισμός συμπληρώθηκε από real-time συγχρονισμό με χρήση SignalR. Έτσι, όταν ένας χρήστης εκτελεί μεταβολή κατάστασης, ο server ειδοποιεί τους υπόλοιπους συνδεδεμένους clients, οι οποίοι ανανεώνουν την εικόνα τους χωρίς χειροκίνητο refresh. Η επιλογή αυτή βελτιώνει σημαντικά τη συνοχή της εφαρμογής σε περιβάλλον πολλαπλών συσκευών.



Εικόνα 4.1: Ροή δεδομένων και συγχρονισμού μεταξύ client, backend και αποθήκευσης

Κεφάλαιο 5 — Μοντελοποίηση Δεδομένων και Σχήμα Βάσης

Η μοντελοποίηση του μενού στη βάση έχει σχεδιαστεί ώστε να είναι απλή αλλά επαρκής για πραγματική χρήση. Η κεντρική οντότητα είναι το MenuItem, με αναγνωριστικό (Id), ονομασία (Name), κατηγορία (Category), τιμή (Price), ενεργοποίηση (Active) και απόθεμα (StockQuantity). Η ύπαρξη του Active επιτρέπει να “κρύβονται” προϊόντα από το PDA UI, όμως στην τελική έκδοση του έργου υιοθετήθηκε και δυνατότητα hard delete μέσω admin, ώστε ο διαχειριστής να μπορεί να αφαιρεί πλήρως προϊόντα από τη βάση όταν το επιθυμεί.

Το απόθεμα υλοποιείται ως StockQuantity τύπου int? (nullable). Αυτό επιτρέπει δύο μοντέλα λειτουργίας χωρίς πρόσθετη πολυπλοκότητα: όταν StockQuantity είναι NULL, το προϊόν θεωρείται “απεριόριστο” και δεν εφαρμόζεται έλεγχος αποθέματος. Όταν είναι ακέραιος αριθμός, το προϊόν έχει πεπερασμένο stock, το οποίο μειώνεται server-side κάθε φορά που γίνεται προσθήκη στο bill. Ο έλεγχος αυτός είναι κρίσιμος, γιατί εξασφαλίζει συνέπεια ανεξάρτητα από το UI και αποτρέπει καταστάσεις όπου δύο συσκευές προσπαθούν να καταναλώσουν το τελευταίο τεμάχιο.

Ο DbContext (BarDbContext) λειτουργεί ως χάρτης μεταξύ κώδικα και βάσης. Σε επίπεδο υλοποίησης, η εφαρμογή ορίζει connection string προς το bar.db και χρησιμοποιεί EF Core για queries, inserts, updates και deletes. Παράλληλα, ο MenuSeeder χρησιμοποιείται ως bootstrap μηχανισμός: ελέγχει αν υπάρχουν δεδομένα και, αν όχι, προσθέτει αρχικά προϊόντα. Αυτή η διαδικασία κάνει την εφαρμογή άμεσα λειτουργική στην πρώτη εκτέλεση.

Στην τελική μορφή του συστήματος, προστέθηκε και “Menu Admin” UI, που λειτουργεί ως πρακτικός μηχανισμός CRUD, χωρίς να απαιτείται άνοιγμα της βάσης σε εξωτερικό εργαλείο. Ο διαχειριστής μπορεί να προσθέτει νέα items, να αλλάζει τιμές, να ενεργοποιεί/απενεργοποιεί προϊόντα, να αλλάζει stock, και να αποθηκεύει όλες τις αλλαγές μαζικά με ένα “Save all”, το οποίο μετατρέπεται σε αντίστοιχα POST/PUT/DELETE requests προς το backend.

Στην τελική έκδοση, η διαχείριση της βάσης δεν περιορίζεται σε αρχικοποίηση μέσω seeding ή σε εξωτερικά εργαλεία, αλλά γίνεται και μέσα από ενσωματωμένο διαχειριστικό περιβάλλον (Menu Admin), το οποίο επιτρέπει προσθήκη, ενημέρωση και διαγραφή εγγραφών του πίνακα MenuItems με πρακτικό τρόπο για τον τελικό χρήστη.

Menu Admin (SQLite)

Edit items and stock. Save writes to bar.db.

+ New item

⌂ Reload

💾 Save all changes

Loaded 202 items

Name	Category	Price	Stock	Active	Actions
🔍 Search by name...					
Alfa Bottle 330ml	Beer	4	0	✓	Delete
Alfa Draft Beer 300ml	Beer	3		✓	Delete
Alfa Draft Beer 500ml	Beer	5		✓	Delete
Amstel Bottle 330ml	Beer	4	41	✓	Delete
Bud Bottle 330ml	Beer	5	17	✓	Delete
Corona Bottle 355ml	Beer	5	44	✓	Delete
Desperados Bottle 330ml	Beer	6	24	✓	Delete
Duvel Bottle 330ml	Beer	6	24	✓	Delete
Erdinger Bottle 500ml	Beer	5	23	✓	Delete
Fix Dark Bottle 330ml	Beer	4	21	✓	Delete
Guinness Can 500ml	Beer	6	24	✓	Delete
Heineken Bottle 330ml	Beer	4	48	✓	Delete
Kopparberg Can 500ml	Beer	6		✓	Delete
Lefte Blonde Bottle 330ml	Beer	6	24	✓	Delete
Lefte Dark Bottle 330ml	Beer	6	24	✓	Delete
Mythos Bottle 330ml	Beer	4	48	✓	Delete

Close

Εικόνα 5.1: Διαχειριστικό περιβάλλον μενού για επεξεργασία προϊόντων και αποθέματος στη βάση SQLite.

Κεφάλαιο 6 — Υλοποίηση Backend: ASP.NET Core Web API και Επιχειρησιακή Λογική

Η υλοποίηση του backend οργανώνεται σε controllers, όπου κάθε controller καλύπτει μια λειτουργική ενότητα. Ο MenuController διαχειρίζεται το μενού, ο TablesController χειρίζεται τραπέζια και λογαριασμούς, και ο RegisterController παρέχει συγκεντρωτική εικόνα ταμείου και εκκρεμοτήτων. Η χρήση JSON με camelCase διασφαλίζει ότι τα πεδία που επιστρέφει το backend ταιριάζουν φυσικά με τη χρήση τους στο JavaScript frontend.

Στον MenuController, τα endpoints διαχωρίζονται σε “PDA view” και “Admin view”. Το PDA endpoint επιστρέφει μόνο ενεργά προϊόντα (Active=true), ώστε το menu grid να εμφανίζει ό,τι είναι διαθέσιμο προς πώληση. Αντίθετα, το admin endpoint επιστρέφει όλα τα προϊόντα (ενεργά και μη), ώστε ο διαχειριστής να μπορεί να τα βλέπει και να τα επεξεργάζεται. Οι αλλαγές από το Admin UI υλοποιούνται μέσω POST (create), PUT (update) και DELETE (hard delete). Το hard delete αφαιρεί οριστικά την εγγραφή από τον πίνακα MenuItem, κάτι που είναι χρήσιμο όταν το κατάστημα θέλει “καθαρή” βάση χωρίς προϊόντα που δεν θα χρησιμοποιηθούν ξανά.

Για την προστασία των διαχειριστικών λειτουργιών προστέθηκε ειδικό endpoint επαλήθευσης κωδικού, μέσω του οποίου το frontend επιτρέπει την είσοδο στο Menu Admin μόνο όταν δοθεί σωστό password. Η λύση αυτή αποτελεί βασικό έλεγχο πρόσβασης, χωρίς ακόμη να επεκτείνεται σε πλήρες σύστημα authentication/authorization.

Στον TablesController, η προσθήκη είδους σε τραπέζι (POST /api/tables/{id}/items) αποτελεί κρίσιμο σημείο καθώς συνδέει το UI με το stock. Το frontend στέλνει menuItemId, και ο server ανακτά το αντίστοιχο MenuItem από SQLite. Αν το προϊόν είναι ανενεργό, απορρίπτει την ενέργεια. Αν έχει πεπερασμένο απόθεμα και αυτό είναι μηδενικό, απορρίπτει την προσθήκη ως out-of-stock. Αν υπάρχει απόθεμα, μειώνει το stock κατά 1 και αποθηκεύει τη μεταβολή στη βάση. Μόνο έπειτα δημιουργεί το item του bill και το προσθέτει στον λογαριασμό του τραπεζιού. Με αυτόν τον τρόπο, η συνέπεια στο stock διασφαλίζεται server-side.

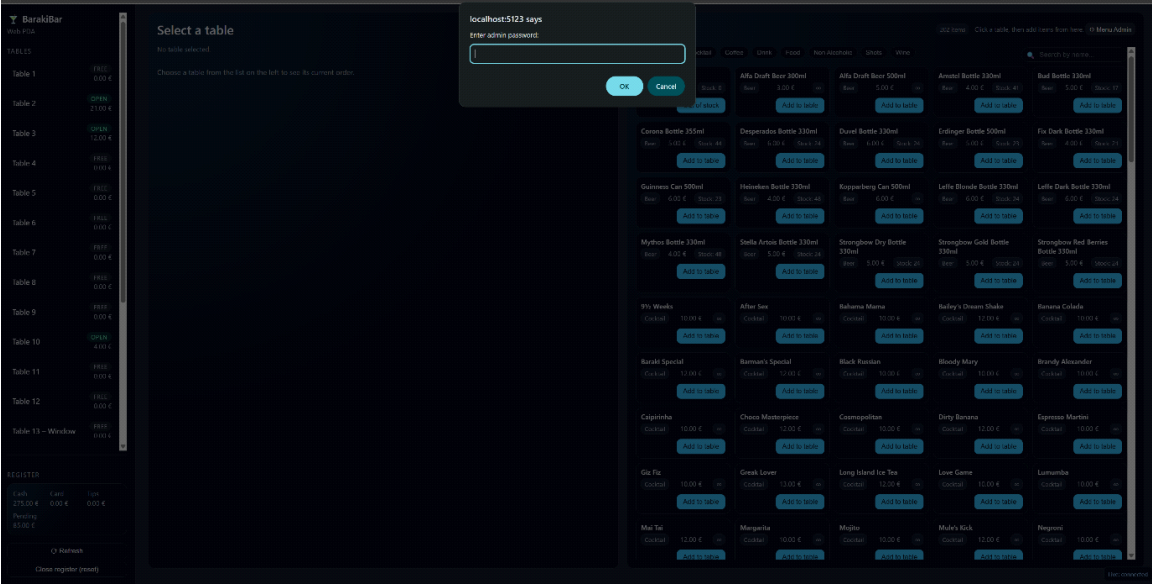
Πέρα από τα REST endpoints, το backend υποστηρίζει και SignalR hub για real-time ενημέρωση των clients. Μετά από μεταβολές όπως προσθήκη item σε τραπέζι, αφαίρεση item, πληρωμή,

αλλαγή προϊόντων στο menu ή reset του register, ο server αποστέλλει μήνυμα προς όλους τους συνδεδεμένους clients ώστε να ανανεώσουν την τοπική απεικόνισή τους.

Η λογική πληρωμών διαχωρίζεται σε ολική και μερική. Η ολική πληρωμή κλείνει τον λογαριασμό, ενημερώνει το register και καθαρίζει το τραπέζι. Η μερική πληρωμή επιτρέπει επιλογή items, δημιουργεί ένα υποσύνολο bill (sub-bill) για υπολογισμό ποσού, ενημερώνει register και αφαιρεί τα πληρωμένα items από το αρχικό bill. Παράλληλα, υλοποιείται και μια επιπλέον πρακτική λειτουργία: αφαίρεση items χωρίς πληρωμή (remove selected). Η λειτουργία “Remove selected” χρησιμοποιείται για διόρθωση λαθών καταχώρισης και αφαίρεση προϊόντων από bill χωρίς πληρωμή. Σε stock-aware περιπτώσεις, η αφαίρεση μπορεί να συνοδεύεται από αντίστοιχη ενημέρωση αποθέματος, ώστε το σύστημα να παραμένει συνεπές ως προς τη λογική διαχείρισης προϊόντων.

Ο RegisterController υπολογίζει cash, card, tips και pending. Το pending αναπαριστά τα ποσά που παραμένουν σε ανοιχτούς λογαριασμούς. Προστέθηκε endpoint “Close register”, το οποίο μηδενίζει cash/card/tips μόνο όταν pending=0. Αυτός ο κανόνας θεωρείται επιχειρησιακά απαραίτητος, καθώς αποτρέπει “κλείσιμο ημέρας” όταν υπάρχουν ακόμη ανοιχτά τραπέζια.

Η αρχικοποίηση του συστήματος (Program.cs) εξασφαλίζει ότι το περιβάλλον είναι έτοιμο πριν εξυπηρετηθούν requests. Πρώτον, γίνεται ρύθμιση controllers και JSON. Δεύτερον, γίνεται ρύθμιση DbContext για SQLite. Τρίτον, εκτελείται seeding όπου χρειάζεται. Επιπλέον, φορτώνεται η κατάσταση bar (tables/register) μέσω FileProcessor πριν το σύστημα ξεκινήσει να δέχεται αιτήματα. Η σωστή σειρά εκκίνησης μειώνει δραστικά errors και εξασφαλίζει σταθερότητα μετά από επανεκκινήσεις.



Εικόνα 6.1: Προστασία πρόσβασης στο διαχειριστικό περιβάλλον μέσω κωδικού.

Κεφάλαιο 7 — Υλοποίηση Frontend: UI Λογική, Απόδοση και API Κλήσεις

Το frontend υλοποιείται ως ελαφριά εφαρμογή χωρίς framework, με καθαρές συναρτήσεις JavaScript για φόρτωση δεδομένων και απόδοση DOM. Κεντρικό ρόλο έχει η `fetchJson()`, η οποία εκτελεί HTTP αιτήσεις και χειρίζεται status codes. Με αυτόν τον τρόπο, όλα τα API calls έχουν κοινή συμπεριφορά στο error handling, κάτι που βελτιώνει τη συντηρησιμότητα.

Η sidebar εμφανίζει τη λίστα τραπεζιών μέσω `loadTables()` (GET /api/tables). Κάθε τραπέζι αποδίδεται με ονομασία, κατάσταση “Open/Free” και σύνολο. Με την επιλογή τραπέζιού, η εφαρμογή αποθηκεύει το `currentTableId` και φορτώνει λεπτομέρειες με `loadTableDetails()` (GET /api/tables/{id}). Εκεί εμφανίζεται αναλυτικός πίνακας items, επιλογές tip και discount, κουμπιά για πληρωμή και λειτουργίες μερικής πληρωμής/αφαίρεσης. Η αφαίρεση items (remove selected) καλύπτει πρακτικά λάθη χειρισμού, που σε πραγματικές συνθήκες εμφανίζονται συχνά, και επιτρέπει στο σύστημα να λειτουργεί χωρίς “τεχνητές λύσεις” όπως έκπτωση 100%.

Το menu view φορτώνεται με `loadMenu()` (GET /api/menu) και εμφανίζεται σε grid. Η δημιουργία φίλτρων γίνεται με `loadMenuCategories()` (GET /api/menu/categories). Στην τελική έκδοση προστέθηκε και search bar στο PDA menu, ώστε η εύρεση προϊόντων να γίνεται γρήγορα με βάση το όνομα. Η αναζήτηση υλοποιείται ως φίλτρο στο `renderMenuGrid()` πάνω στο array `menuItems`, χωρίς επιπλέον κλήσεις στο backend, κάτι που είναι επαρκές για μικρό μενού και μειώνει network traffic.

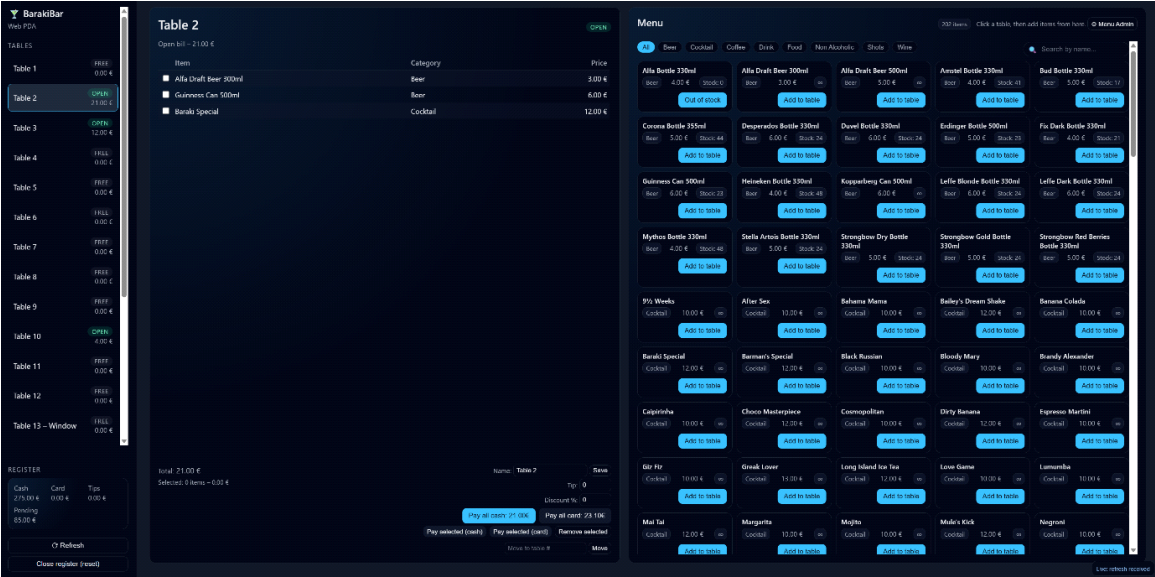
Παράλληλα, ενσωματώθηκε modal “Menu Admin”, το οποίο λειτουργεί ως πλήρες περιβάλλον διαχείρισης του μενού. Το admin UI φορτώνει όλα τα προϊόντα (GET /api/menu/all), επιτρέπει επεξεργασία πεδίων, αναζήτηση μέσω δεύτερου search bar και υποστηρίζει “Save all changes”. Η υλοποίηση κρατά προσωρινές αλλαγές σε δομές (`adminEdits`, `adminDeletedIds`) και όταν ο χρήστης πατήσει “Save all”, η εφαρμογή μεταφράζει τις αλλαγές σε αντίστοιχα POST/PUT/DELETE requests. Η επιλογή μαζικής αποθήκευσης μειώνει την πιθανότητα μερικής/ασυνεπούς κατάστασης, καθώς ο διαχειριστής ολοκληρώνει τις αλλαγές σε ένα commit βήμα.

Στην τελική μορφή της εφαρμογής δόθηκε ιδιαίτερη προσοχή στη mobile εμπειρία. Για μικρές οθόνες προστέθηκε responsive προσαρμογή της διάταξης, καθώς και ειδική navigation bar που επιτρέπει άμεση μετάβαση μεταξύ της λίστας τραπεζιών, της προβολής παραγγελίας και του μενού. Επιπλέον, η επιλογή τραπεζιού σε mobile διάταξη μπορεί να συνοδεύεται από αυτόματη μεταφορά της προβολής στο αντίστοιχο section, ώστε να μειώνεται η αίσθηση “χάους” κατά το scrolling.

Ιδιαίτερο ενδιαφέρον για τεκμηρίωση αποτελεί και το θέμα caching στον browser, το οποίο εμφανίστηκε κατά την ανάπτυξη ως πρακτικό πρόβλημα. Σε web εφαρμογές, είναι πιθανό ο browser να κρατά cached εκδόσεις του app.js ή του index.html. Αυτό μπορεί να οδηγήσει σε φαινομενικά “ανεξήγητες” περιπτώσεις όπου ο προγραμματιστής έχει αφαιρέσει UI στοιχεία ή έχει διορθώσει HTML, αλλά το UI εξακολουθεί να εμφανίζει την παλιά μορφή. Στο project, η επιβεβαίωση έγινε με άνοιγμα σε διαφορετικό browser ή hard refresh. Επιπλέον, ένα πραγματικό σφάλμα που εμφανίστηκε ήταν η απουσία συγκεκριμένων IDs στο HTML (π.χ. #menu-grid), η οποία προκαλούσε τη σιωπηρή αποτυχία απόδοσης του μενού (renderMenuGrid() επέστρεφε νωρίς επειδή δεν έβρισκε το element). Αυτή η εμπειρία υπογραμμίζει ότι στη web ανάπτυξη η συνέπεια μεταξύ HTML δομής και JavaScript selectors είναι κρίσιμη.

Σε επίπεδο frontend, η σύνδεση με το SignalR hub επιτρέπει την αυτόματη ανανέωση του UI όταν άλλος client μεταβάλει την κατάσταση του συστήματος. Έτσι, πολλαπλές συσκευές μπορούν να παραμένουν συγχρονισμένες χωρίς χειροκίνητο refresh.

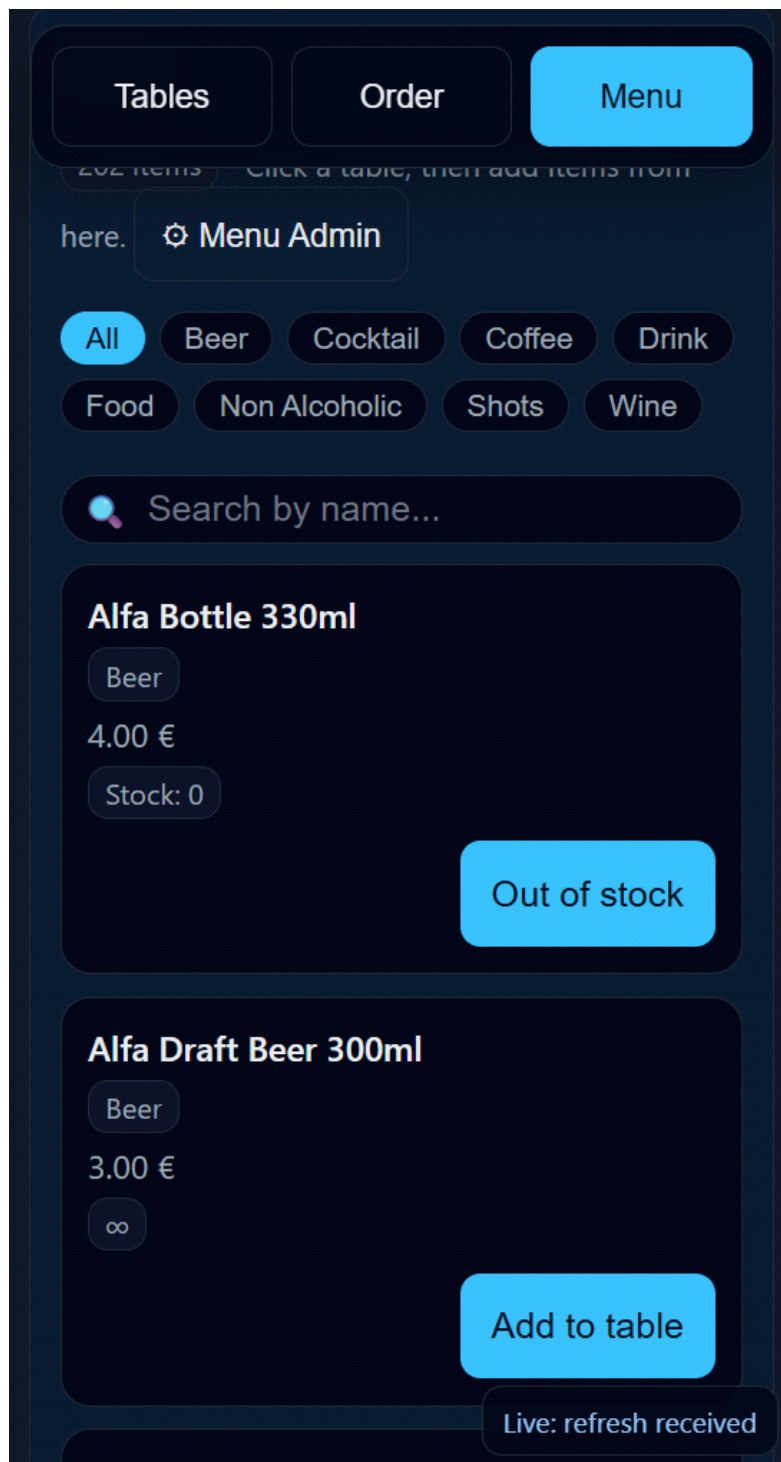
Στην τελική βελτίωση της mobile εμπειρίας, η navigation bar διατηρείται συνεχώς ορατή κατά το scrolling και ενημερώνει οπτικά το ενεργό section (τραπέζια, παραγγελία ή μενού), ώστε ο χρήστης να μπορεί να προσανατολίζεται άμεσα μέσα στη σελίδα.



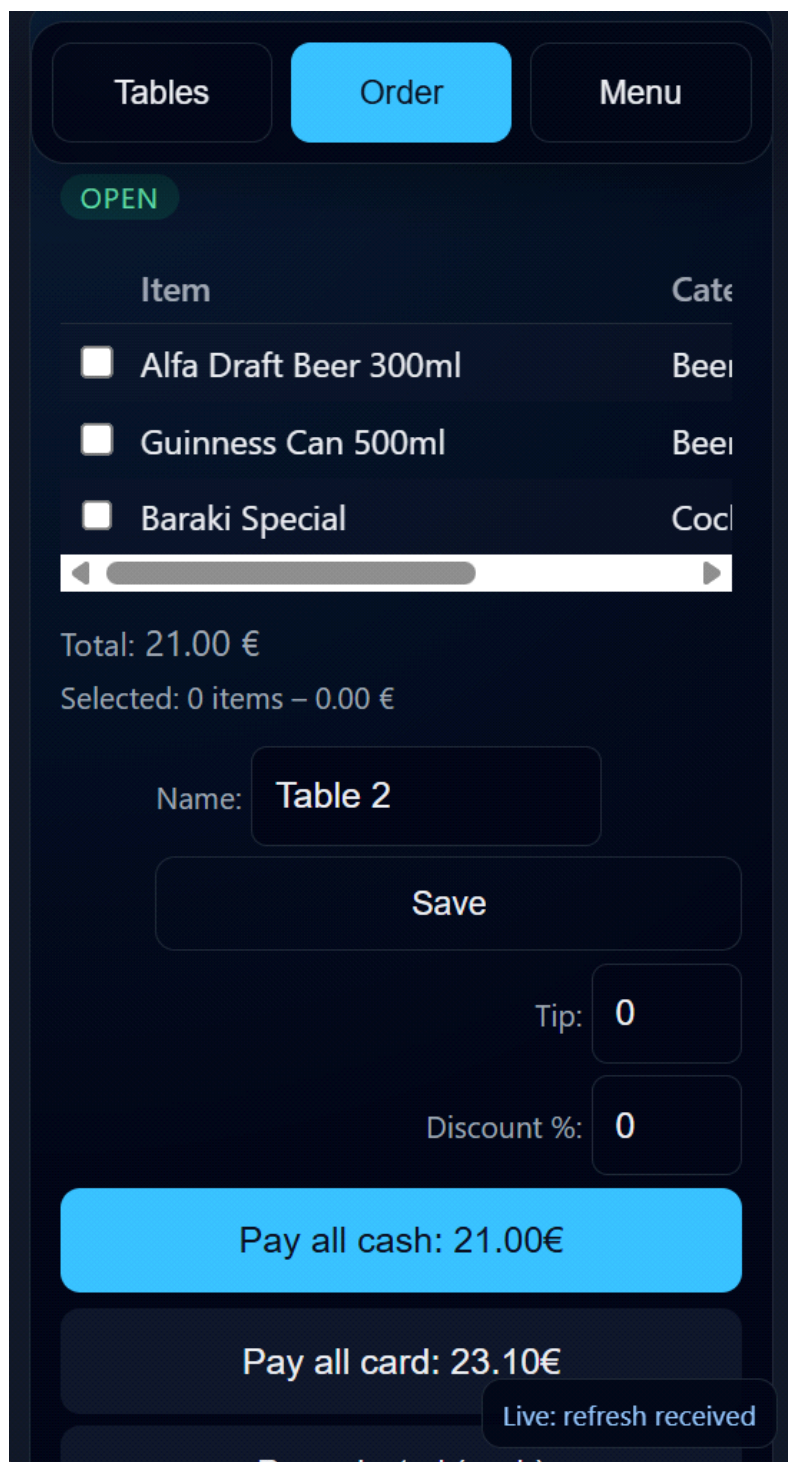
Εικόνα 7.1: Η κύρια διεπαφή της εφαρμογής σε desktop περιβάλλον.



Εικόνα 7.2: Η διεπαφή της εφαρμογής σε κινητή συσκευή.



Εικόνα 7.3: Mobile navigation bar για γρήγορη μετάβαση μεταξύ τραπεζιών, παραγγελίας και μενού.



Εικόνα 7.4: Προβολή μενού με φίλτρα και αναζήτηση προϊόντων.

Κεφάλαιο 8 — Δοκιμές, Σενάρια Χρήσης και Επαλήθευση Λειτουργικότητας

Η αξιολόγηση ενός τέτοιου συστήματος γίνεται κυρίως με σενάρια λειτουργικής επαλήθευσης που προσομοιώνουν πραγματική χρήση σε bar. Ένα βασικό σενάριο είναι η επιλογή τραπέζιού, η προσθήκη ειδών και η επιβεβαίωση ότι το τραπέζι γίνεται “open” και ότι το sidebar εμφανίζει σωστό σύνολο. Στη συνέχεια εκτελείται ολική πληρωμή και ελέγχεται ότι το register ενημερώνεται σωστά (cash ή card, tips) και ότι το τραπέζι καθαρίζει ή αφαιρείται (στην περίπτωση bar tables) σύμφωνα με τους κανόνες.

Ένα δεύτερο σενάριο αφορά μερική πληρωμή. Ο χρήστης επιλέγει items μέσω checkbox, εκτελεί “pay selected”, και ελέγχει ότι αφαιρούνται μόνο τα επιλεγμένα, ενώ το υπόλοιπο bill παραμένει. Ο δείκτης pending πρέπει να μειωθεί αναλογικά. Παράλληλα, τα tips πρέπει να αυξάνονται ανεξάρτητα από τον τρόπο πληρωμής.

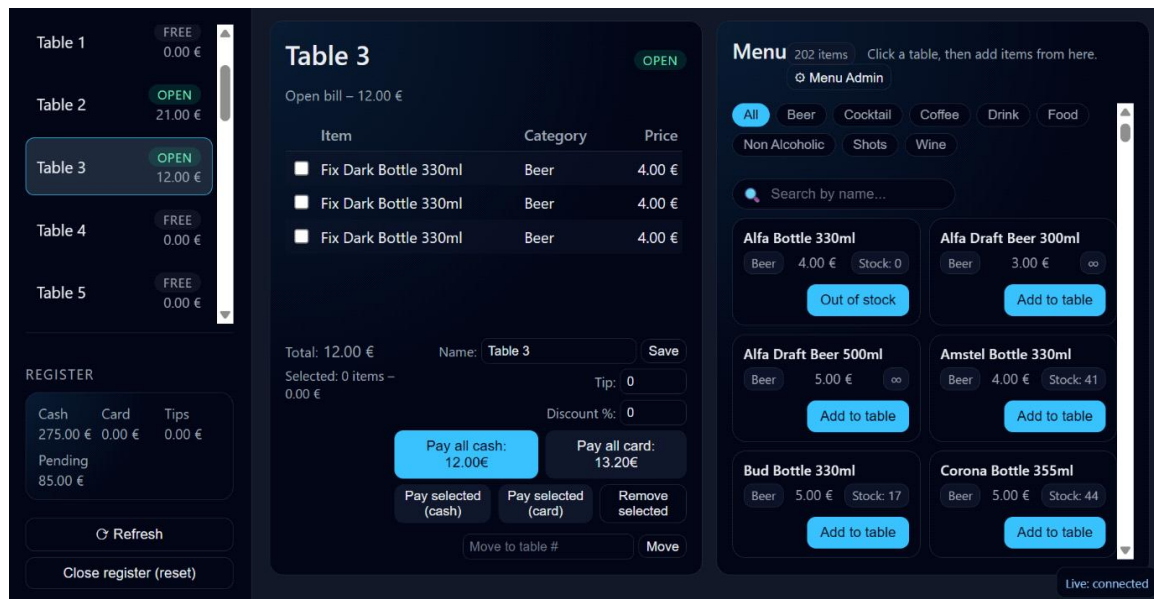
Ένα τρίτο σενάριο αφορά το stock. Για προϊόν με stock 1, η πρώτη προσθήκη πρέπει να επιτρέπεται και να μειώνει το stock σε 0, ενώ δεύτερη προσθήκη πρέπει να απορρίπτεται server-side. Ο client πρέπει να εμφανίζει μήνυμα αποτυχίας και να ανανεώνει το menu ώστε να φαίνεται το “Out of stock”. Ένα πρακτικό σενάριο είναι η ακύρωση λάθους παραγγελίας: αφαιρώντας items από το τραπέζι χωρίς πληρωμή, επιβεβαιώνεται ότι ο λογαριασμός διορθώνεται. Σε stock-aware υλοποίηση, ελέγχεται και η επιστροφή stock όταν αφαιρείται προϊόν περιορισμένης ποσότητας.

Για την επαλήθευση persistence, γίνεται restart του server και επιβεβαιώνεται ότι το μενού παραμένει στη SQLite (bar.db). Επιπλέον, γίνεται έλεγχος του “Close register”: η εφαρμογή πρέπει να επιτρέπει reset του register μόνο όταν pending=0, κάτι που επαληθεύεται ανοίγοντας τραπέζι, αυξάνοντας pending, και δοκιμάζοντας να κλείσει το register (αναμένεται απόρριψη).

Πραγματοποιήθηκε έλεγχος ταυτόχρονης χρήσης της εφαρμογής από διαφορετικούς clients, χρησιμοποιώντας δύο browsers και κινητή συσκευή στο ίδιο τοπικό δίκτυο. Επιβεβαιώθηκε ότι όταν ένας χρήστης εκτελεί μεταβολή, όπως προσθήκη ή αφαίρεση item, οι υπόλοιποι clients ενημερώνονται σε πραγματικό χρόνο και ανανεώνουν το UI μέσω SignalR.

Η εφαρμογή δοκιμάστηκε επίσης σε smartphone μέσω browser, χρησιμοποιώντας πρόσβαση από το ίδιο δίκτυο Wi-Fi με τον server. Η δοκιμή έδειξε ότι το σύστημα μπορεί να χρησιμοποιηθεί λειτουργικά και από κινητή συσκευή, ενώ οι responsive προσαρμογές και η mobile navigation bar βελτιώνουν αισθητά τη χρηστικότητα σε μικρή οθόνη.

Κατά τη φάση δοκιμών διαπιστώθηκε επίσης ότι το browser caching μπορεί να επηρεάσει την ορθή παρατήρηση αλλαγών στο frontend, ειδικά σε αρχεία όπως app.js. Για τον λόγο αυτό, μέρος των δοκιμών επαληθεύτηκε και με hard refresh ή με χρήση δεύτερου browser, ώστε να αποκλειστεί η πιθανότητα ψευδών εντυπώσεων λόγω παλαιών cached εκδόσεων του UI.



Εικόνα 8.1: Δοκιμή της εφαρμογής από κινητή συσκευή στο ίδιο τοπικό δίκτυο.

Κεφάλαιο 9 — Συμπεράσματα, Περιορισμοί και Μελλοντικές Επεκτάσεις

Το BarakiBar Web PDA αποδεικνύει ότι μια σύγχρονη web προσέγγιση μπορεί να καλύψει λειτουργικές ανάγκες ενός μικρού καταστήματος με ελάχιστη υποδομή. Το ASP.NET Core παρέχει σταθερό backend με καθαρά APIs και δυνατότητες επέκτασης. Η SQLite προσφέρει πρακτική επίμονη αποθήκευση χωρίς εξωτερικό server, ενώ το EF Core απλοποιεί την πρόσβαση στα δεδομένα. Το frontend παραμένει λειτουργικό και διαχειρίσιμο χωρίς framework, με σωστό διαχωρισμό φόρτωσης/απόδοσης και ξεκάθαρο μοτίβο ανανέωσης δεδομένων μετά από κάθε μεταβολή. Η τελική μορφή του έργου περιλαμβάνει όχι μόνο βασική παραγγελιοληψία και διαχείριση λογαριασμών, αλλά και διαχειριστικό περιβάλλον, έλεγχο αποθέματος, real-time συγχρονισμό μεταξύ πολλαπλών clients και mobile-friendly προσαρμογή για χρήση σε πραγματικές συνθήκες.

Σημαντική βελτίωση στην τελική έκδοση είναι η ενσωμάτωση διαχειριστικού περιβάλλοντος για το μενού (Menu Admin), με δυνατότητα αναζήτησης και μαζικής αποθήκευσης αλλαγών στη βάση. Αυτό μειώνει την εξάρτηση από εξωτερικά εργαλεία και κάνει το σύστημα πρακτικότερο για πραγματική λειτουργία. Παράλληλα, προστέθηκαν κανόνες ασφαλείας όπως το “Close register μόνο όταν pending=0”, που ευθυγραμμίζονται με πραγματικές διαδικασίες ταμείου.

Παρόλα αυτά, υπάρχουν περιορισμοί που αξίζει να επισημανθούν. Η σημαντικότερη τεχνική παρατήρηση αφορά τη διπλή αποθήκευση: μέρος της κατάστασης διατηρείται σε αρχεία μέσω FileProcessor, ενώ το μενού/stock στη βάση. Αυτό δημιουργεί δύο “πηγές αλήθειας” και αυξάνει τη δυσκολία πλήρους ενοποίησης της κατάστασης. Σε επόμενη φάση θα ήταν ωφέλιμο να μεταφερθούν και τα tables/bills/register σε κοινό data layer στη βάση, ώστε το σύστημα να γίνει πλήρως persistent και ευκολότερο στη συντήρηση.

Παρότι στην τελική έκδοση προστέθηκε βασική προστασία πρόσβασης στο Menu Admin μέσω κωδικού, η εφαρμογή δεν διαθέτει ακόμη πλήρες σύστημα authentication και authorization με ρόλους χρηστών. Σε μελλοντική έκδοση θα μπορούσε να υποστηριχθεί σαφέστερος διαχωρισμός μεταξύ χειριστή και διαχειριστή.

ΠΑΡΑΡΤΗΜΑ Α — Ενδεικτικά API Endpoints και Παραδείγματα Κλήσεων (REST)

A.1 Γενικές αρχές επικοινωνίας

Η εφαρμογή BarakiBar Web PDA ακολουθεί αρχές επικοινωνίας τύπου REST μεταξύ του frontend (browser) και του backend (ASP.NET Core Web API). Το frontend εκτελεί HTTP κλήσεις προς endpoints που εκτίθενται από controllers και λαμβάνει απαντήσεις σε μορφή JSON. Για λόγους συμβατότητας με JavaScript, το JSON χρησιμοποιεί ονοματοδοσία camelCase (π.χ. menuItemId, stockQuantity, paymentMethod). Οι ενέργειες που δεν χρειάζεται να επιστρέψουν περιεχόμενο συχνά απαντούν με HTTP 204 (No Content). Τα σφάλματα επιστρέφουν κωδικούς όπως 400 (Bad Request) ή 404 (Not Found), με μήνυμα που μπορεί να εμφανιστεί στον χρήστη.

Σημαντική αρχή σχεδιασμού είναι ότι το UI δεν θεωρείται “πηγή αλήθειας”. Για παράδειγμα, ο έλεγχος stock δεν βασίζεται μόνο στο αν το κουμπί είναι disabled στο UI, αλλά γίνεται οριστικά στο backend μέσω SQLite/EF Core. Αντίστοιχα, κανόνες όπως “Close register μόνο όταν pending=0” εφαρμόζονται στον server και όχι μόνο στο frontend.

A.2 Tables API

A.2.1 Λίστα τραπεζιών

Μέθοδος: GET

Διαδρομή: /api/tables

Περιγραφή: Επιστρέφει όλα τα τραπέζια (σταθερά και δυναμικά “bar tables”), μαζί με την κατάσταση (open/free) και το σύνολο bill.

Ενδεικτική απάντηση (200 OK):

```
[
```

```
{
```

```
  "id": 1,
```

```
  "name": "",
```

```
  "open": false,
```

```
  "total": 0.0,
```

```
"items": []
},
{
  "id": 15,
  "name": "Giorgos",
  "open": true,
  "total": 9.0,
  "items": [
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 },
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 }
  ]
}
```

A.2.2 Λεπτομέρειες συγκεκριμένου τραπεζιού

Μέθοδος: GET

Διαδρομή: /api/tables/{id}

Παράδειγμα: GET /api/tables/3

Ενδεικτική απάντηση (200 OK):

```
{
  "id": 3,
  "name": "Nikos",
  "open": true,
  "total": 12.5,
  "items": [
    { "name": "Espresso", "category": "Coffee", "price": 2.5 },
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 },
```

```
{ "name": "Water", "category": "Soft", "price": 1.0 }
```

```
]
```

```
}
```

Ενδεικτική απάντηση (404 Not Found):

Table 999 not found.

A.2.3 Δημιουργία νέου bar table

Μέθοδος: POST

Διαδρομή: /api/tables

Περιγραφή: Δημιουργεί νέο “bar table” (ID > 14). Τα σταθερά τραπέζια 1–14 θεωρούνται μόνιμα.

Ενδεικτική απάντηση (201 Created):

```
{
```

```
  "id": 16,
```

```
  "name": "Bar 2",
```

```
  "total": 0.0,
```

```
  "open": false
```

```
}
```

A.2.4 Ονομασία τραπεζιού (όνομα πελάτη)

Μέθοδος: POST

Διαδρομή: /api/tables/{id}/name

Ενδεικτικό αίτημα:

```
{ "name": "Giorgos" }
```

Απάντηση: 204 No Content

A.2.5 Προσθήκη είδους στο τραπέζι (με έλεγχο αποθέματος)

Μέθοδος: POST

Διαδρομή: /api/tables/{id}/items

Περιγραφή: Ο client στέλνει menuItemId (Id από τη βάση). Το backend ανακτά το MenuItem από SQLite, ελέγχει active και stockQuantity, μειώνει το stock όταν χρειάζεται και προσθέτει το item στο bill.

Ενδεικτικό αίτημα:

```
{ "menuItemId":3 }
```

Ενδεικτική απάντηση (200 OK):

```
{  
  "id": 3,  
  "name": "Nikos",  
  "open": true,  
  "total": 15.0,  
  "items": [  
    { "name": "Espresso", "category": "Coffee", "price": 2.5 },  
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 },  
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 },  
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 }  
  ]  
}
```

Ενδεικτική απάντηση (400 Bad Request):

Out of stock.

A.2.6 Κλείσιμο τραπεζιού (ολική πληρωμή)

Μέθοδος: POST

Διαδρομή: /api/tables/{id}/close

Περιγραφή: Δέχεται paymentMethod (“cash” ή “card”), tip και discountPercent. Σε πληρωμή κάρτας μπορεί να εφαρμόζεται multiplier (π.χ. 1.1) σύμφωνα με κανόνα του συστήματος.

Ενδεικτικό αίτημα:


```
{  
  "paymentMethod": "cash",  
  "tip": 1.0,  
  "discountPercent": 10  
}
```

Απάντηση: 204 No Content

Ενδεικτική απάντηση (400 Bad Request):

PaymentMethod must be 'cash' or 'card'.

A.2.7 Μερική πληρωμή επιλεγμένων ειδών

Μέθοδος: POST

Διαδρομή: /api/tables/{id}/pay-items

Περιγραφή: Ο client στέλνει λίστα από itemIndexes (θέσεις items στον τρέχοντα λογαριασμό) μαζί με paymentMethod και tip. Το backend υπολογίζει ποσό, ενημερώνει register, καταγράφει ιστορικό και αφαιρεί τα πληρωμένα items από το bill.

Ενδεικτικό αίτημα:

```
{  
  "paymentMethod": "card",  
  "tip": 0.5,  
  "itemIndexes": [0, 2]  
}
```

Ενδεικτική απάντηση (200 OK):

```
{  
  "id": 3,  
  "name": "Nikos",  
  "open": true,  
  "total": 8.0,  
  "items": [  
    {  
      "id": 1,  
      "name": "Coke",  
      "price": 1.0,  
      "open": true,  
      "total": 1.0,  
      "itemIndexes": [0]  
    },  
    {  
      "id": 2,  
      "name": "Fries",  
      "price": 2.0,  
      "open": true,  
      "total": 2.0,  
      "itemIndexes": [2]  
    },  
    {  
      "id": 3,  
      "name": "Burger",  
      "price": 3.0,  
      "open": true,  
      "total": 3.0,  
      "itemIndexes": [1]  
    }  
  ]  
}
```

```
{ "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 },  
{ "name": "Water", "category": "Soft", "price": 1.0 },  
{ "name": "Water", "category": "Soft", "price": 1.0 },  
{ "name": "Water", "category": "Soft", "price": 1.5 }  
]  
}
```

A.2.8 Αφαίρεση επιλεγμένων ειδών (χωρίς πληρωμή / ακύρωση)

Μέθοδος: POST

Διαδρομή: /api/tables/{id}/remove-items

Περιγραφή: Αφαιρεί επιλεγμένα items από το bill χωρίς να ενημερώνει ταμείο.

Χρησιμοποιείται για διόρθωση λαθών παραγγελίας.

Ενδεικτικό αίτημα:

```
{ "itemIndexes": [1,3] }
```

Ενδεικτική απάντηση (200 OK):

```
{  
  "id": 3,  
  "name": "Nikos",  
  "open": true,  
  "total": 10.0,  
  "items": [  
    { "name": "Espresso", "category": "Coffee", "price": 2.5 },  
    { "name": "Draft Beer 500ml", "category": "Beer", "price": 4.5 },  
    { "name": "Water", "category": "Soft", "price": 3.0 }  
  ]  
}
```

A.2.9 Μεταφορά λογαριασμού σε άλλο τραπέζι

Μέθοδος: POST

Διαδρομή: /api/tables/{id}/move-to-table

Ενδεικτικό αίτημα:

```
{ "targetTableId": 5 }
```

Απάντηση: 204 No Content

A.3 Menu API (SQLite μέσω EF Core)

A.3.1 Λίστα ενεργών ειδών (PDA Menu)

Μέθοδος: GET

Διαδρομή: /api/menu

Περιγραφή: Επιστρέφει μόνο προϊόντα με active=true (αυτά που εμφανίζονται στο PDA). Το πεδίο index αντιστοιχεί στο Id της βάσης.

Ενδεικτική απάντηση (200 OK):

```
[
```

```
{
```

```
  "index": 1,
```

```
  "name": "Espresso",
```

```
  "category": "Coffee",
```

```
  "price": 2.5,
```

```
  "active": true,
```

```
  "stockQuantity": null
```

```
},
```

```
{
```

```
  "index": 2,
```

```
  "name": "Draft Beer 500ml",
```

```
  "category": "Beer",
```

```
"price": 4.5,  
"active": true,  
"stockQuantity": 50  
}  
]
```

A.3.2 Λίστα όλων των ειδών (Admin Menu)

Μέθοδος: GET

Διαδρομή: /api/menu/all

Περιγραφή: Επιστρέφει όλα τα προϊόντα (και ανενεργά), ώστε να μπορούν να διαχειριστούν από το Menu Admin UI.

A.3.3 Λίστα κατηγοριών

Μέθοδος: GET

Διαδρομή: /api/menu/categories

Ενδεικτική απάντηση (200 OK):

```
["Beer", "Coffee", "Soft"]
```

A.3.4 Προσθήκη νέου item

Μέθοδος: POST

Διαδρομή: /api/menu

Ενδεικτικό αίτημα:

```
{  
  "name": "Tonic Water",  
  "category": "Soft",  
  "price": 2.2,  
  "active": true,  
  "stockQuantity": 24  
}
```

Ενδεικτική απάντηση (201 Created):

```
{  
  "index": 10,  
  "name": "Tonic Water",  
  "category": "Soft",  
  "price": 2.2,  
  "active": true,  
  "stockQuantity": 24  
}
```

A.3.5 Ενημέρωση item (τιμή/stock/active)

Μέθοδος: PUT

Διαδρομή: /api/menu/{id}

Ενδεικτικό αίτημα:

```
{  
  "price": 4.8,  
  "stockQuantity": 40,  
  "active": true  
}
```

A.3.6 Hard delete item (οριστική διαγραφή)

Μέθοδος: DELETE

Διαδρομή: /api/menu/{id}

Περιγραφή: Διαγράφει οριστικά το προϊόν από τη βάση (bar.db).

Απάντηση: 204 No Content

A.4 Register API

A.4.1 Κατάσταση ταμείου και εκκρεμοτήτων

Μέθοδος: GET

Διαδρομή: /api/register

Ενδεικτική απάντηση (200 OK):

```
{  
  "cash": 25.0,  
  "card": 33.0,  
  "pending": 18.5,  
  "tips": 4.0  
}
```

A.4.2 Close register (reset με προϋπόθεση pending=0)

Μέθοδος: POST

Διαδρομή: /api/register/close

Περιγραφή: Επιτρέπει reset cash/card/tips μόνο όταν pending == 0. Αν υπάρχουν εκκρεμότητες, επιστρέφει σφάλμα και το reset δεν πραγματοποιείται.

A.4.3 Επαλήθευση πρόσβασης στο Admin UI

Μέθοδος: POST

Διαδρομή: /api/admin/verify

Περιγραφή:

Ελέγχει τον κωδικό πρόσβασης για είσοδο στο διαχειριστικό περιβάλλον του μενού.

Ενδεικτικό αίτημα:

```
{  
  "password": "1234"  
}
```

Ενδεικτική απάντηση (200 OK):

```
{  
  "success": true  
}
```

Ενδεικτική απάντηση (401 Unauthorized):

Wrong password.

ΠΑΡΑΡΤΗΜΑ Β — SQLite / EF Core: Σχήμα, Παραδείγματα SQL και Ερμηνεία Αποτελεσμάτων

B.1 Τοπική αποθήκευση με SQLite

Η SQLite χρησιμοποιείται ως τοπική βάση δεδομένων σε μορφή αρχείου (bar.db). Η εφαρμογή την αξιοποιεί για επίμονη αποθήκευση του μενού και του αποθέματος (stock), ώστε οι αλλαγές να παραμένουν μετά από επανεκκίνηση. Η σύνδεση ορίζεται με connection string τύπου Data Source=.../bar.db. Αυτό αποφεύγει την ανάγκη ξεχωριστού database server, μειώνει την πολυπλοκότητα εγκατάστασης και επιτρέπει εύκολη μεταφορά/backup του αρχείου βάσης.

B.2 Entity MenuItem και αντιστοίχιση στον πίνακα MenuItems

Η βασική οντότητα που αποθηκεύεται στη SQLite είναι το MenuItem, με πεδία: Id (Primary Key), Name, Category, Price, Active και StockQuantity. Η σχεδίαση χρησιμοποιεί nullable StockQuantity. Όταν είναι NULL, το προϊόν θεωρείται “απεριόριστο” και δεν εφαρμόζεται έλεγχος αποθέματος. Όταν είναι ακέραιος αριθμός, το προϊόν έχει πεπερασμένο stock και μειώνεται στο backend κάθε φορά που προστίθεται σε τραπέζι.

B.3 Ενδεικτικό DDL δημιουργίας πίνακα

Ανάλογα με migrations/EnsureCreated, ο πίνακας MenuItems αντιστοιχεί εννοιολογικά σε σχήμα όπως:

```
CREATE TABLE MenuItems (  
  Id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
  Name TEXT NOT NULL,  
  Category TEXT NOT NULL,  
  Price NUMERIC NOT NULL,  
  Active INTEGER NOT NULL,  
  StockQuantity INTEGER NULL  
);
```

Σημείωση για Price: Η SQLite χρησιμοποιεί δυναμική τυποποίηση και δεν παρέχει αυστηρό “decimal” τύπο όπως ο SQL Server. Το EF Core χαρτογραφεί το Price ώστε στο επίπεδο εφαρμογής να χρησιμοποιείται decimal και οι υπολογισμοί χρημάτων να εκτελούνται με ακρίβεια στο backend.

B.4 Πρακτικά SQL queries για έλεγχο και αναφορές

- Εμφάνιση όλων των προϊόντων:

```
SELECT Id, Name, Category, Price, Active, StockQuantity
```

```
FROM MenuItems
```

```
ORDER BY Category, Name;
```

- Εμφάνιση μόνο ενεργών προϊόντων (όπως στο PDA UI):

```
SELECT Id, Name, Category, Price, StockQuantity
```

```
FROM MenuItems
```

```
WHERE Active = 1
```

```
ORDER BY Category, Name;
```

- Εντοπισμός προϊόντων εκτός αποθέματος:

```
SELECT Id, Name, Category, StockQuantity
```

```
FROM MenuItems
```

```
WHERE Active = 1 AND StockQuantity = 0;
```

- Εντοπισμός “απεριόριστων” προϊόντων:

```
SELECT Id, Name, Category
```

```
FROM MenuItems
```

```
WHERE Active = 1 AND StockQuantity IS NULL;
```

- Χειροκίνητη διόρθωση αποθέματος:

```
UPDATE MenuItems
```

```
SET StockQuantity = 25
```

```
WHERE Id = 2;
```


- Απενεργοποίηση προϊόντος (απόκρυψη από PDA χωρίς διαγραφή):

```
UPDATE MenuItems
```

```
SET Active = 0
```

```
WHERE Id = 10;
```

- Οριστική διαγραφή προϊόντος (hard delete):

```
DELETE FROM MenuItems
```

```
WHERE Id = 10;
```

B.5 Ερμηνεία EF Core queries και ροή ενημέρωσης stock

Στο backend, το EF Core χρησιμοποιεί LINQ για να μεταφράζει C# εκφράσεις σε SQL. Για την προβολή του μενού, εφαρμόζεται φίλτρο `Active=true`. Για την προσθήκη προϊόντος σε τραπέζι με stock control, το backend ακολουθεί ροή `read-check-update`. Αρχικά ανακτά το MenuItem από τη βάση βάσει Id. Στη συνέχεια ελέγχει αν είναι Active και αν το StockQuantity είναι μηδέν. Αν υπάρχει διαθέσιμο stock (ή αν είναι NULL), επιτρέπεται η προσθήκη. Όταν το StockQuantity είναι αριθμός, μειώνεται κατά 1 και εκτελείται `SaveChangesAsync()` ώστε η μεταβολή να αποθηκευτεί στη SQLite. Μετά από αυτό, δημιουργείται το αντίστοιχο item στο bill του τραπεζίου.

Η επιλογή server-side ελέγχου είναι κρίσιμη: ακόμη κι αν το UI εμφανίζει σωστά “Out of stock” και απενεργοποιεί κουμπιά, η τελική απόφαση γίνεται στο backend, άρα το σύστημα παραμένει ορθό ακόμη και σε caching, πολλαπλές συσκευές ή χειροκίνητες κλήσεις προς endpoints.

B.6 Διαγνωστικός έλεγχος σωστής λειτουργίας της βάσης

Για να επιβεβαιωθεί ότι η βάση λειτουργεί, ελέγχεται ότι ο πίνακας MenuItems υπάρχει, ότι περιέχει εγγραφές μετά το seeding και ότι οι τιμές αλλάζουν μετά από CRUD ενέργειες μέσω Menu Admin. Ένα χαρακτηριστικό σφάλμα είναι το “no such table: MenuItems”, το οποίο δείχνει ότι το schema δεν δημιουργήθηκε πριν από τα πρώτα queries. Η ορθή πρακτική είναι η δημιουργία/εφαρμογή schema (`EnsureCreated` ή `migrations`) πριν εκτελεστούν `AnyAsync/CountAsync/Select`.

Στην τελική έκδοση, η καθημερινή διαχείριση της βάσης γίνεται κυρίως από το ενσωματωμένο Menu Admin UI, ενώ τα παραπάνω SQL παραδείγματα λειτουργούν κυρίως ως διαγνωστικά και τεκμηριωτικά εργαλεία.

ΠΑΡΑΡΤΗΜΑ Γ — Τελική Ροή Χρήσης και Διαχείριση Μενού/Αποθέματος στην Τελική Έκδοση

Γ.1 Περιγραφή τελικής εμπειρίας χρήσης (end-to-end)

Η τελική έκδοση του BarakiBar Web PDA έχει σχεδιαστεί ώστε να καλύπτει πλήρως τις καθημερινές ανάγκες ενός bar σε επίπεδο παραγγελιοληψίας, διαχείρισης τραπεζιών, κλεισίματος λογαριασμών και ελέγχου αποθέματος. Ο χειριστής ανοίγει την εφαρμογή από οποιονδήποτε σύγχρονο browser και, χωρίς εγκατάσταση, αποκτά πρόσβαση σε ένα περιβάλλον τύπου PDA, κατάλληλο για χρήση από κινητό, tablet ή υπολογιστή.

Στην αριστερή στήλη εμφανίζεται η λίστα τραπεζιών. Υπάρχουν σταθερά τραπέζια (1–14) και επιπλέον δυναμικά “bar tables” που δημιουργούνται κατά απαίτηση, όταν εξυπηρετούνται πελάτες στον πάγκο. Κάθε τραπέζι εμφανίζει συνοπτικά την κατάσταση (Open/Free) και το τρέχον σύνολο. Η επιλογή ενός τραπεζιού φορτώνει αναλυτικά τα στοιχεία του λογαριασμού: λίστα ειδών, συνολικό ποσό, επιλογές πληρωμής (ολικής ή μερικής), καθώς και δυνατότητα μεταφοράς λογαριασμού σε άλλο τραπέζι.

Στη δεξιά στήλη εμφανίζεται το μενού προϊόντων, οργανωμένο σε κατηγορίες και σε μορφή καρτών. Ο χειριστής μπορεί να χρησιμοποιήσει φίλτρα κατηγορίας και αναζήτηση (search bar) για άμεσο εντοπισμό προϊόντος. Με την ενέργεια “Add to table”, το προϊόν προστίθεται στο επιλεγμένο τραπέζι και το σύστημα ενημερώνει αυτόματα το σύνολο του λογαριασμού, την κατάσταση του τραπεζιού (από Free σε Open), το “pending” στο register, καθώς και το stock, όταν το προϊόν έχει περιορισμένη ποσότητα.

Η εφαρμογή καλύπτει πρακτικές ανάγκες καθημερινής λειτουργίας, όπως η μερική πληρωμή (split bill) και η διόρθωση λάθους παραγγελίας. Σε περίπτωση που προστέθηκαν λάθος είδη, ο χειριστής μπορεί να επιλέξει συγκεκριμένες γραμμές μέσω checkbox και να χρησιμοποιήσει “Remove selected” ώστε τα είδη να αφαιρεθούν από το bill χωρίς πληρωμή. Αντίστοιχα, για μερική πληρωμή, ο χειριστής επιλέγει συγκεκριμένα items και εκτελεί “Pay selected”, ώστε να πληρωθεί μόνο το επιλεγμένο υποσύνολο, ενώ τα υπόλοιπα παραμένουν ανοικτά στο ίδιο τραπέζι.

Το register στο κάτω μέρος της αριστερής στήλης εμφανίζει συγκεντρωτικά ποσά για cash, card, tips και pending. Στην τελική έκδοση, το “Close register (reset)” προστατεύεται με κανόνα

ασφαλείας: επιτρέπεται μόνο όταν το pending είναι μηδέν, ώστε να μην μπορεί να γίνει reset ενώ υπάρχουν ανοικτές οφειλές. Αυτό διασφαλίζει ότι η μηδενική εκκρεμότητα αποτελεί προϋπόθεση πριν μηδενιστούν τα συγκεντρωτικά ποσά.

Γ.2 Τελική αρχιτεκτονική ροή δεδομένων (UI → API → SQLite/State → UI)

Η τελική υλοποίηση ακολουθεί σταθερό μοτίβο συγχρονισμού. Το frontend λειτουργεί ως client που απεικονίζει δεδομένα και εκτελεί ενέργειες μέσω API calls. Το backend αποτελεί την “πηγή αλήθειας”, καθώς εφαρμόζει τους κανόνες και διατηρεί τα επίμονα δεδομένα. Σε κάθε ενέργεια που μεταβάλλει κατάσταση (mutation), το UI δεν βασίζεται σε “local υπολογισμούς” για να μαντέψει την τελική εικόνα, αλλά ακολουθεί την πρακτική της επαναφόρτωσης των βασικών τμημάτων που επηρεάζονται. Έτσι αποφεύγονται ασυνέπειες που θα εμφανίζονταν αν ο client επιχειρούσε να συγχρονίσει χειροκίνητα πολλαπλές τιμές.

Το σύστημα χρησιμοποιεί δύο διακριτά επίπεδα αποθήκευσης, με σαφή ρόλο το καθένα. Το μενού και το απόθεμα αποθηκεύονται στη SQLite βάση bar.db μέσω Entity Framework Core, προσφέροντας ανθεκτικότητα και μόνιμη αποθήκευση. Η κατάσταση των τραπεζιών και των λογαριασμών (bill state), όπως και το register, διατηρούνται ως runtime κατάσταση στον server (BarState) και αποθηκεύονται επίσης στο αντίστοιχο state file μέσω FileProcessor, ώστε να παραμένει η λειτουργική κατάσταση μετά από restart. Η διάκριση αυτή στην τελική έκδοση είναι συνειδητή: το μενού/stock απαιτεί καθαρό CRUD και ιστορικό αλλαγών, ενώ ο λογαριασμός τραπεζιού είναι λειτουργική κατάσταση που μεταβάλλεται πολύ συχνά κατά τη διάρκεια μιας βάρδιας.

Η πιο κρίσιμη ροή της εφαρμογής είναι η “stock-aware” προσθήκη προϊόντος σε τραπέζι. Το UI στέλνει στο backend μόνο το menuItemId και όχι τιμή/όνομα ως δεδομένο αλήθειας. Το backend ανακτά το αντίστοιχο MenuItem από SQLite, ελέγχει Active και StockQuantity, μειώνει το stock όταν απαιτείται, και στη συνέχεια δημιουργεί το item στο bill του τραπεζιού με τα αξιόπιστα δεδομένα της βάσης. Με αυτόν τον τρόπο, ακόμη και αν το UI είναι cached ή αν υπάρχουν πολλαπλοί clients, ο server παραμένει ο τελικός ελεγκτής συνέπειας.

Γ.3 Τελική διαχείριση μενού και αποθέματος μέσω Admin UI (Save All, αναζήτηση, hard delete)

Στην τελική έκδοση, η διαχείριση του μενού υλοποιείται ως ενσωματωμένο “Menu Admin” περιβάλλον (modal) μέσα στο ίδιο web app. Ο στόχος είναι η πλήρης CRUD διαχείριση της βάσης bar.db χωρίς εξωτερικά εργαλεία και χωρίς να απαιτείται χειροκίνητη SQL. Το Admin UI εμφανίζει όλα τα προϊόντα (ενεργά και ανενεργά), επιτρέπει επεξεργασία πεδίων (name, category, price, stock, active) και παρέχει αναζήτηση με ξεχωριστό search bar για ταχύτερο φιλτράρισμα.

Κρίσιμο στοιχείο της τελικής εμπειρίας είναι ότι οι αλλαγές δεν αποθηκεύονται ανά προϊόν, αλλά συγκεντρώνονται σε ένα “working set” και καταγράφονται με ένα μόνο κουμπί: “Save all changes”. Το UI διατηρεί τις αλλαγές σε προσωρινή δομή (edits/deletes) και κατά το Save εκτελεί τις απαραίτητες κλήσεις προς το backend. Αυτό έχει πρακτικά πλεονεκτήματα: ο χρήστης μπορεί να αλλάξει πολλά προϊόντα, να προσθέσει νέα, να σημειώσει διαγραφές, και να επιβεβαιώσει όλα μαζί με μία ενέργεια, μειώνοντας τον χρόνο και τα λάθη.

Η διαγραφή προϊόντος στην τελική έκδοση είναι hard delete. Δηλαδή, όταν ένα προϊόν επιλεγεί για διαγραφή και αποθηκευτούν οι αλλαγές, αφαιρείται οριστικά από τον πίνακα MenuItems στη SQLite. Η επιλογή hard delete έχει καθαρή συμπεριφορά και απλοποιεί τη διαχείριση, ειδικά σε μικρά συστήματα όπου δεν απαιτείται ιστορικό “αδρανοποίησης” προϊόντων. Παρ’ όλα αυτά, παραμένει διαθέσιμο και το πεδίο Active για περιπτώσεις όπου ο διαχειριστής θέλει να αποκρύψει προσωρινά προϊόντα από το PDA menu χωρίς να τα διαγράψει.

Τέλος, για τον χρήστη του PDA, η ύπαρξη ξεχωριστής αναζήτησης στο βασικό menu (frontend page) επιτρέπει γρήγορη εξυπηρέτηση, ενώ η αναζήτηση στο Admin UI απευθύνεται στη διαχειριστική χρήση. Η ύπαρξη δύο search bars σε διαφορετικά contexts θεωρείται καλή πρακτική UI, καθώς επιλύει διαφορετικά προβλήματα: “βρες προϊόν για παραγγελία” έναντι “βρες προϊόν για διαχείριση/διόρθωση”.

ΠΑΡΑΡΤΗΜΑ Δ — Τεχνική Επαλήθευση, Troubleshooting και Έλεγχοι Ορθής Λειτουργίας

Δ.1 Έλεγχος συγχρονισμού UI και API (consistency checks)

Η τελική έκδοση βασίζεται σε μοτίβο “reload-after-mutation”. Μετά από ενέργειες όπως add item, remove selected, pay selected και close table, το UI πρέπει να ανανεώνει τουλάχιστον τα παρακάτω όπου απαιτείται: table details, tables list, register, και menu (για stock). Η σωστή επαναφόρτωση εξασφαλίζει ότι το UI δεν εμφανίζει παλιά δεδομένα ή λάθος totals.

Ενδεικτική επαλήθευση: προσθήκη προϊόντος με stock αριθμητικό (π.χ. 2). Μετά από δύο “Add to table” το UI πρέπει να εμφανίζει stock=0 και να απενεργοποιεί το κουμπί προσθήκης.

Παράλληλα, τρίτη προσπάθεια προσθήκης πρέπει να απορρίπτεται από τον server με μήνυμα “Out of stock.”, ακόμη και αν ο client επιχειρήσει χειροκίνητη κλήση.

Δ.2 Έλεγχος λειτουργίας SQLite (schema, seeding, CRUD)

Για να επιβεβαιωθεί η ορθή λειτουργία της SQLite βάσης, απαιτούνται τρεις βασικοί έλεγχοι: ύπαρξη αρχείου bar.db, ύπαρξη του πίνακα MenuItems και ύπαρξη δεδομένων (seeded ή δημιουργημένων από Admin UI). Ένας απλός επιβεβαιωτικός έλεγχος είναι μέσω endpoint debug (π.χ. /debug/state) ή μέσω λειτουργίας “Menu Admin” (που φορτώνει /api/menu/all). Αν το Admin UI εμφανίζει items, τότε η βάση είναι λειτουργική και περιέχει δεδομένα.

Επιπλέον, το CRUD επαληθεύεται πρακτικά ως εξής: προσθήκη νέου item από Admin, αλλαγή τιμής σε υπάρχον, αλλαγή stock, και hard delete. Μετά από κάθε Save all, το “Reload” στο Admin πρέπει να αντικατοπτρίζει τις αλλαγές και το PDA menu (/api/menu) πρέπει να εμφανίζει μόνο τα ενεργά items.

Δ.3 Caching σε browser και αποφυγή “φανταστικών” προβλημάτων UI

Κατά την ανάπτυξη web εφαρμογών, συχνό πρόβλημα είναι η cache του browser (ιδίως για app.js). Αυτό μπορεί να οδηγήσει σε συμπεριφορά όπου το UI δεν συμφωνεί με τον τρέχοντα κώδικα, εμφανίζοντας παλιές συναρτήσεις ή διαφορετικό DOM. Η τελική διαδικασία αντιμετώπισης περιλαμβάνει hard refresh, δοκιμή σε δεύτερο browser και, όπου απαιτείται, χρήση cache: “no-store” σε ευαίσθητες κλήσεις (όπως στο Admin load).

Ενδεικτικό σύμπτωμα caching: το Admin UI εμφανίζει σωστά προϊόντα αλλά το PDA menu εμφανίζει κενό. Σε τέτοια περίπτωση ελέγχεται πρώτα ότι το menu-grid στοιχείο υπάρχει στο HTML και ότι η render function καλείται σωστά. Έπειτα ελέγχεται ότι το endpoint /api/menu επιστρέφει ενεργά προϊόντα (Active=true). Αν όλα είναι σωστά αλλά το UI παραμένει κενό, η πιθανότερη αιτία είναι cached JS.

Δ.4 Έλεγχος σωστής διάκρισης /api/menu vs /api/menu/all

Στην τελική έκδοση υπάρχουν δύο διαφορετικές ροές: το PDA menu χρησιμοποιεί /api/menu (μόνο ενεργά items), ενώ το Admin UI χρησιμοποιεί /api/menu/all (όλα τα items). Ένας συχνός έλεγχος ορθότητας είναι να επιβεβαιωθεί ότι το UI καλεί το σωστό endpoint. Αν το PDA menu είναι κενό αλλά το Admin εμφανίζει προϊόντα, τότε πιθανότατα όλα τα προϊόντα είναι Active=false ή το frontend δεν καλεί σωστά το /api/menu.

Δ.5 Έλεγχος κανόνα “Close register μόνο όταν pending=0”

Η τελική έκδοση περιλαμβάνει λειτουργικό κανόνα που αποτρέπει reset του register όταν υπάρχουν ανοικτές οφειλές. Η επαλήθευση γίνεται δημιουργώντας ανοικτό τραπέζι με bill και επιβεβαιώνοντας ότι το pending είναι >0. Σε αυτή την κατάσταση, το “Close register (reset)” πρέπει να μπλοκάρει την ενέργεια (με μήνυμα/απόρριψη). Αφού κλείσουν όλα τα τραπέζια και μηδενιστεί το pending, τότε επιτρέπεται reset.

Δ.6 Έλεγχος ακύρωσης/διόρθωσης παραγγελίας (Remove selected)

Για λειτουργική ακρίβεια σε πραγματικό περιβάλλον, απαιτείται δυνατότητα διόρθωσης λαθών. Η τελική έκδοση υποστηρίζει “Remove selected” για να αφαιρούνται items από bill χωρίς πληρωμή. Η επαλήθευση γίνεται με προσθήκη πολλών items σε τραπέζι, επιλογή μερικών, εκτέλεση remove, και έλεγχο ότι το σύνολο μειώνεται ακριβώς κατά το άθροισμα των αφαιρεθέντων γραμμών.

Δ.7 Έλεγχος real-time συγχρονισμού πολλών clients

Ο έλεγχος real-time συγχρονισμού πραγματοποιήθηκε με ταυτόχρονη πρόσβαση στην εφαρμογή από δύο διαφορετικούς browsers και από κινητή συσκευή στο ίδιο τοπικό δίκτυο.

Επιβεβαιώθηκε ότι όταν ένας client εκτελεί μεταβολή κατάστασης, όπως προσθήκη προϊόντος, αφαίρεση item ή αλλαγή στο μενού, οι υπόλοιποι συνδεδεμένοι clients λαμβάνουν ειδοποίηση μέσω SignalR και ανανεώνουν αυτόματα την τοπική απεικόνιση χωρίς χειροκίνητο refresh. Η δοκιμή αυτή επαληθεύει ότι το σύστημα μπορεί να χρησιμοποιηθεί ταυτόχρονα από περισσότερους του ενός χειριστές.

Δ.8 Έλεγχος responsive/mobile λειτουργίας και mobile navigation bar

Η responsive/mobile λειτουργία ελέγχθηκε σε smartphone σε portrait και landscape mode, καθώς και σε διαφορετικούς browsers. Επιβεβαιώθηκε ότι η διεπαφή παραμένει λειτουργική σε μικρές οθόνες, ότι η mobile navigation bar επιτρέπει γρήγορη μετάβαση μεταξύ τραπεζιών, παραγγελίας και μενού, και ότι η εμπειρία χρήσης παραμένει πρακτική ακόμη και με έντονο scrolling. Ιδιαίτερη έμφαση δόθηκε στη διατήρηση της ορατότητας της mobile navigation bar και στη σωστή επισήμανση του ενεργού section, ώστε ο χρήστης να μην χάνει τον προσανατολισμό του μέσα στη σελίδα.